

Nehal Garg

Los Angeles, CA | +1 510-815-4322 | nehalgarg2901@gmail.com | [Github](#) | [Linkedin](#) | [Portfolio](#)

SUMMARY

Generalist full-stack Computer Science MS graduate, equally comfortable reasoning about a SQL schema and building the UI on top of it. Production experience spans Python/Django, Node.js/TypeScript, React, and Go, with strong relational data modeling and hands-on work building permissions and access-control systems (RBAC, multi-layer auth, audit logging). Builds daily with Claude and other AI coding tools to prototype and ship faster, while personally testing and reviewing every change. Energized by variety and picking up whatever stack a project calls for, rather than specializing in a single layer.

TECHNICAL SKILLS

Languages: Python, TypeScript, JavaScript, Java, Go, SQL

Data & Relational Modeling: PostgreSQL, MongoDB, Relational Schema Design, Query Optimization

Back-End & Admin Tooling: Django, Django REST Framework, Node.js, Express, FastAPI, REST APIs, RBAC, Audit Logging

Front-End: React, Next.js, React Native, HTML5, CSS3

AI-Native Tooling & Cloud: Claude Code, OpenAI API, Claude API, Agentic Tool-Calling, AWS, GCP, Docker, CI/CD, Git

EXPERIENCE

ML/AI Engineering Intern

Aug 2026 - Present

Allyvia LLC (Python, Django, Django REST Framework, PostgreSQL, Stripe API, QuickBooks API)

Los Angeles, CA (Remote)

- A production QuickBooks Online integration was silently corrupting financial data, so I traced the root cause through the Django/DRF and PostgreSQL layer and shipped a fix that stopped duplicate accounting entries from reaching real SMB customers.
- Financial API writes had no safety net against network failures, so I built idempotent retry safeguards and over 40 automated tests, then delivered further integration features independently, scoping work directly with the founders.

Software Engineer Intern

June 2025 - Sept 2025

ZofAI (Python, TypeScript, PostgreSQL, GCP, Docker, CI/CD)

San Francisco, CA

- 5+ enterprise clients needed a consistent data platform to depend on, so I designed relational data models and shipped typed REST APIs in Python and TypeScript, into production every single week without a missed release.
- Real query patterns were driving slow response times at scale, so I restructured PostgreSQL schemas and indexes around them, cutting response times by approximately 30% and shipping every change through a CI/CD pipeline on GCP.

Software Engineer Intern

Sept 2023 - Nov 2023

Essell 24 (TypeScript, React.js, REST APIs, Git, CI/CD)

San Francisco, CA

- A page-load performance issue was hurting retention on a consumer product with thousands of daily active users, so I traced it to unoptimized asset loading in the React front end and shipped a fix using lazy loading and compression, reducing latency by 16%.
- Regressions were becoming a real risk as release frequency climbed, so I built and unit-tested reusable React/TypeScript components for every change I shipped, catching issues in CI/CD via Git before they ever reached production.

Software Developer Intern

June 2023 - Aug 2023

Hammer Lifestyle (React Native, Redux, Node.js, Firebase, REST APIs, MongoDB)

Haryana, India

- Frequent sync failures were undermining a real-time health-tracking app used by 10K+ active smartwatch users, so I redesigned the data-sync architecture against Firebase, cutting failures by 24% and restoring reliable real-time tracking.
- Slow MongoDB queries were causing a data-ingestion bottleneck, so I partnered with backend engineers to debug and optimize the queries, clearing the backlog and restoring reliable delivery to downstream analytics.

Software Developer Intern

March 2023 - May 2023

Nyadeko (React.js, TypeScript, Node.js, MongoDB, JWT)

New Delhi, India

- Response times were degrading under concurrent load, so I traced the bottleneck to inefficient server-side logic, refactored with async handling, and cut API response latency by 35%.
- Session handling lacked persistent, secure management, so I implemented RESTful endpoints in Node.js with token-based, multi-layer authentication, closing a gap that risked surfacing as a production security issue.

PROJECTS

Ledgerly - Admin Dashboard & Reconciliation Tool (Next.js, TypeScript, Node.js, Prisma, PostgreSQL, REST APIs) [\[Live\]](#)

- Finance teams had no fast way to reconcile bank transactions against internal records, so I designed and shipped Ledgerly alone, a full-stack admin dashboard with role-based permissions and a full audit log, still running in production today.
- Trust in the numbers mattered as much as speed, so I built auto-matching logic that flags mismatches and routes every suggested match through a mandatory human-review step before it's approved and permanently recorded.

Hospital Management System (React.js, GoLang, PostgreSQL, REST APIs, JWT) [\[GitHub\]](#)

- Patient records were scattered across a fragmented, ad hoc system, so I built a full-stack platform in GoLang and React with REST APIs and role-based access control, ensuring only authorized clinicians could touch sensitive records.
- As more departments came online, redundant database queries started slowing the platform, so I audited real query patterns and normalized the PostgreSQL schemas, cutting redundant queries by 25% while keeping data consistent at scale.

AiroLite - Agentic Tool-Calling Platform (Next.js, TypeScript, Node.js, Express, PostgreSQL, Prisma, Claude API) [\[GitHub\]](#)

- Coordinating decisions from scattered live data was a messy, real-world operations problem, so I solo-built AiroLite: a tool-calling agent that reasons over the data and responds to users conversationally through a React-based front end.
- Shipping fast without sacrificing quality mattered from day one, so I used Claude Code and other AI coding tools throughout the build while writing tests and reviewing every generated suggestion myself, then owned CI/CD deployment and on-call debugging solo.

GitStage - Retrieval & Semantic Search Pipeline (Python, FastAPI, React, OpenAI API, pgvector, Redis, Docker) [\[Live\]](#)

- Scattered, unstructured data needed to be searchable and trustworthy, so I built a Python/FastAPI backend with semantic chunking, embeddings, and pgvector similarity search powered directly by the OpenAI API, with a React front end.
- Long-running data jobs needed to stay fast and reliable at scale, so I built a multi-service architecture with async Celery workers and Redis caching on Docker Compose, then owned deployment and monitoring of the live service myself.

RESEARCH PUBLICATION

AppleGo - Product Discovery Platform (React.js, Django, MySQL, Google Maps API, REST APIs) [\[Publication Link\]](#)

- Analyzed 50K+ product queries to identify a gap in real-time retailer visibility for local product search, and proposed a scalable React/Django backend using the Google Maps API for real-time inventory comparison across retailers.

Blockchain-based Crowdfunding Framework (IJISRT) (Solidity, Node.js, PostgreSQL, REST APIs, Git) [\[Publication Link\]](#)

- Traditional crowdfunding platforms lacked transparency, so I proposed a Solidity smart-contract framework designed to make every transaction auditable and secure without any centralized intermediary.

EDUCATION

Masters of Science in Computer Science

May 2026

University of Southern California

Courses: Data Structures & Algorithms, Applied NLP, Machine Learning for Data Science, Software Engineering, Web Technologies

Bachelor of Technology in Computer Science

June 2024

Guru Gobind Singh Indraprastha University

Courses: Data Structures, Object Oriented Programming, Operating Systems, Artificial Intelligence, Distributed and Parallel Systems